

Probabilistic Programming

Marius Popescu

popescunmarius@gmail.com

2025 - 2026

Variational Bayesian Methods

Variational Bayesian Methods

- Variational Bayesian methods are a family of techniques for approximating *intractable integrals* arising in Bayesian inference and machine learning. They are typically used in complex statistical models consisting of observed variables (usually termed "data") as well as unknown parameters and latent variables, with various sorts of relationships among the three types of random variables.
- Variational Bayesian methods are primarily used for two purposes:
 - To provide an analytical approximation to the posterior probability of the unobserved variables, in order to do statistical inference over these variables.
 - To derive a lower bound for the marginal likelihood (sometimes called the "evidence") of the observed data (i.e. the marginal probability of the data given the model, with marginalization performed over unobserved variables). This is typically used for performing model selection, the general idea being that a higher marginal likelihood for a given model indicates a better fit of the data by that model and hence a greater probability that the model in question was the one that generated the data.

Variational Inference

Approximate inference

Probabilistic model: $p(x, \theta) = p(x|\theta)p(\theta)$

Variational Inference

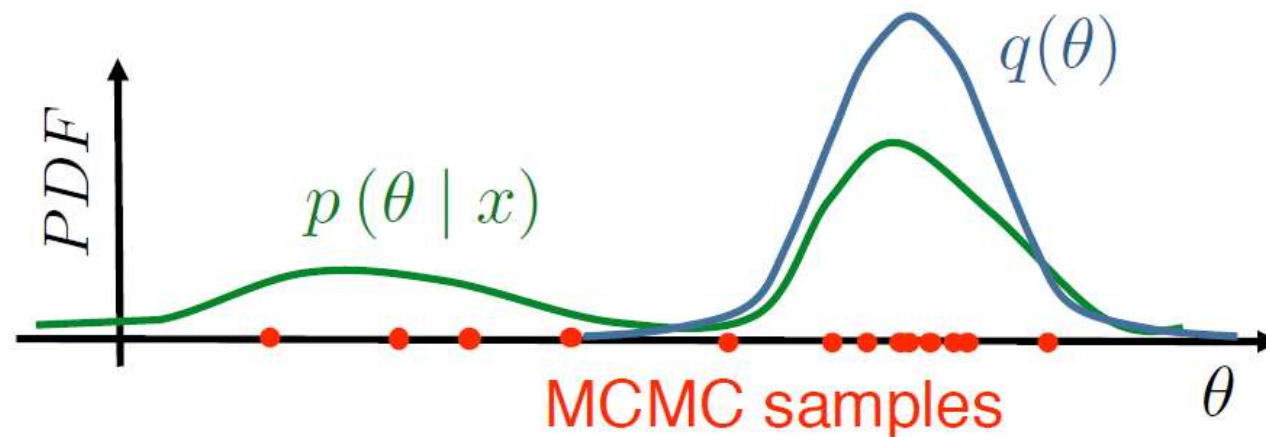
Approximate $p(\theta|x) \approx q(\theta) \in \mathcal{Q}$

- Biased
- Faster and more scalable

MCMC

Samples from unnormalized $p(\theta|x)$

- Unbiased
- Need a lot of samples



Variational Inference

In variational inference, the posterior distribution over a set of unobserved variables $\mathbf{Z} = \{Z_1, \dots, Z_n\}$ given some data $\mathbf{X}$ is approximated by a variational distribution, $q(\mathbf{Z})$:

$$p(\mathbf{Z}|\mathbf{X}) \approx q(\mathbf{Z})$$

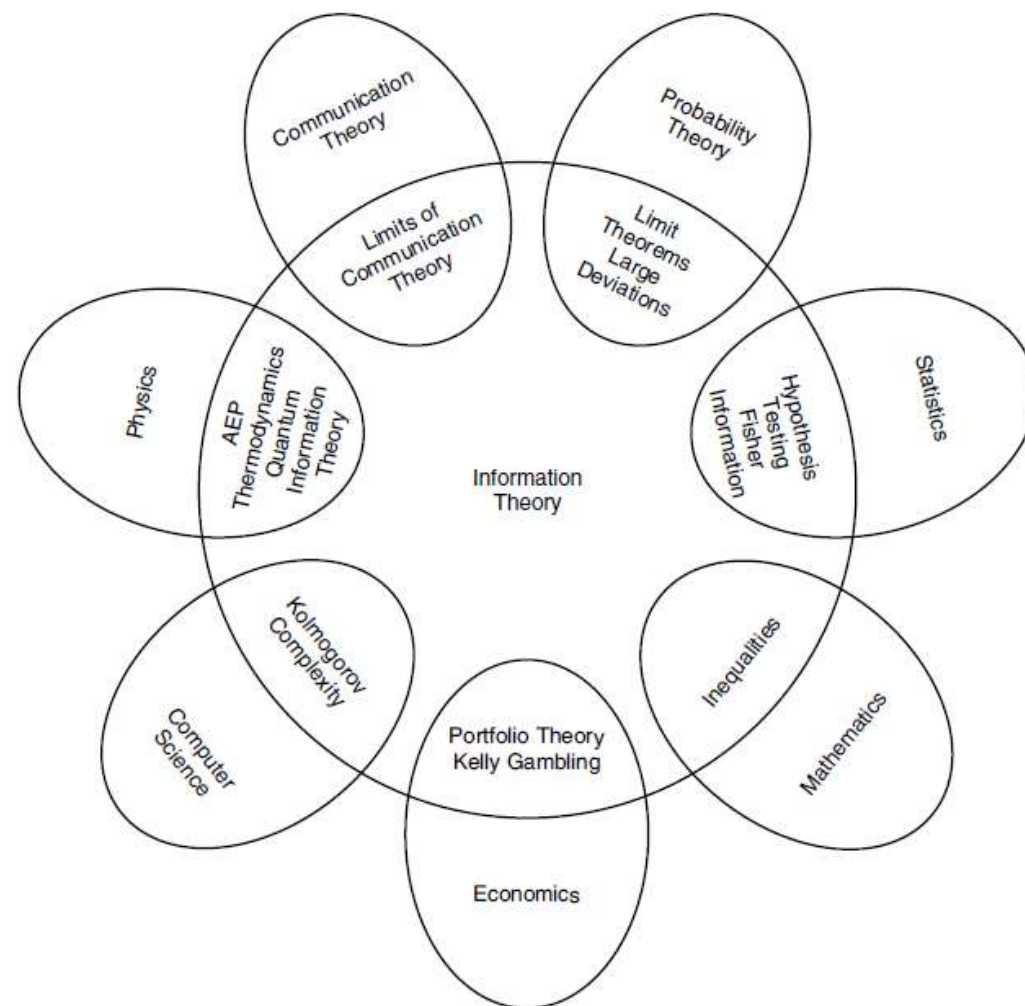
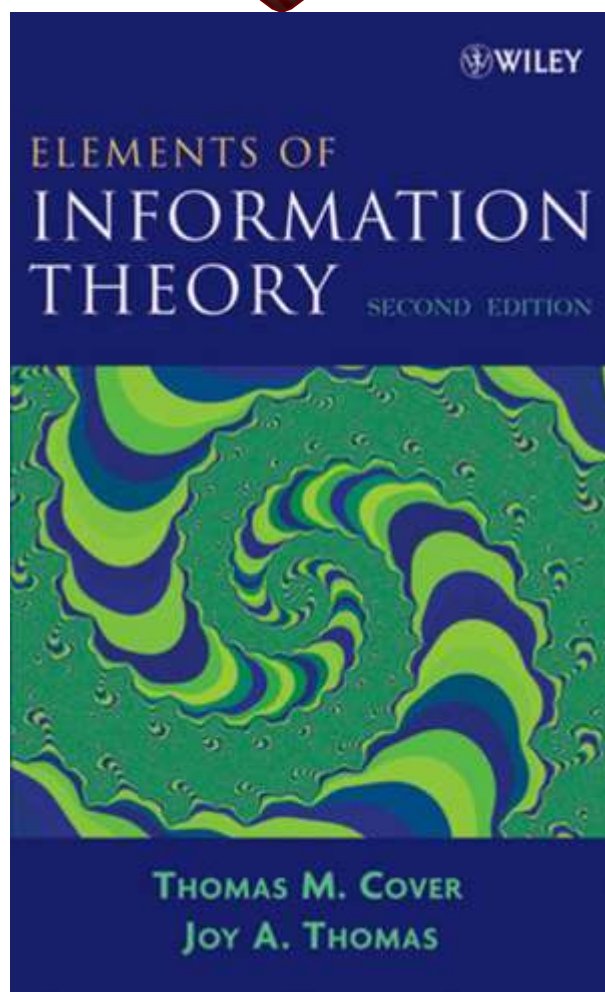
The distribution $q(\mathbf{Z})$ is restricted to belong to a family of distributions (with its own *variational parameters* λ) of simpler form than $p(\mathbf{Z}|\mathbf{X})$ (typically, the true posterior is not in the variational family).

The aim is to find the variational parameters λ that makes q close to the posterior of interest and use q with the fitted parameters as a proxy for the posterior, e.g., to form predictions about future data or to investigate the posterior distribution of the hidden variables.

The most common type of variational inference uses the Kullback–Leibler divergence (KL-divergence) of the two distributions as measure of the closeness.

Variational inference can be seen as an extension of the maximum a posteriori estimation (MAP) of the single most probable value of each parameter to fully Bayesian estimation which computes (an approximation to) the entire posterior distribution of the latent variables.

KL-divergence comes from Information Theory



Information Theory



In the early 1940s it was thought to be impossible to send information at a positive rate with negligible probability of error. Shannon surprised the communication theory community by proving that the probability of error could be made nearly zero for all communication rates below channel capacity. The capacity can be computed simply from the noise characteristics of the channel. Shannon further argued that random processes such as music and speech have an irreducible complexity below which the signal cannot be compressed. This he named *the entropy*, in deference to the parallel use of this word in thermodynamics, and argued that if the entropy of the source is less than the capacity of the channel, asymptotically error-free communication can be achieved.

Information theory answers two fundamental questions in communication theory: What is the ultimate data compression (answer: the entropy H), and what is the ultimate transmission rate of communication (answer: the channel capacity C).

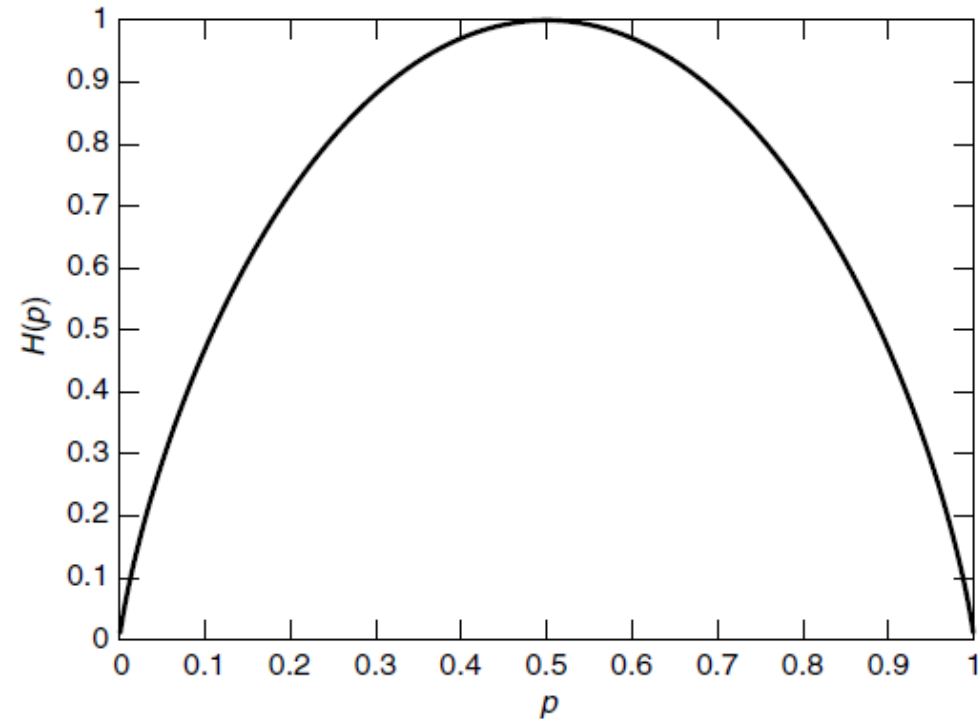
The Entropy

The entropy $H(X)$ of a discrete random variable X is defined by:

$$H(X) = - \sum_{x \in X} p(x) \log_2 p(x) = E_p \log_2 \frac{1}{p(x)}$$

Intuitively, the entropy $H(X)$ of a discrete random variable X is a measure of the amount of uncertainty associated with the value of X when only its distribution is known; it is a measure of the amount of information required on the average to describe the random variable (we could construct a code with average description length $H(X)$).

The entropy is expressed in bits. For example, the entropy of a fair coin toss is 1 bit.



Kullback–Leibler Divergence

For discrete probability distributions p and q defined on the same probability space, the KL-divergence between p and q is defined as:

$$D_{KL}(p||q) = - \sum_{x \in X} p(x) \log_2 \frac{q(x)}{p(x)} = \sum_{x \in X} p(x) \log_2 \frac{p(x)}{q(x)}$$

For distributions p and q of a continuous random variable, the KL-divergence is defined to be the integral:

$$D_{KL}(p||q) = \int_{-\infty}^{\infty} p(x) \log_2 \frac{p(x)}{q(x)} dx$$

More generally:

$$D_{KL}(p||q) = E_p \log_2 \frac{p(x)}{q(x)}$$

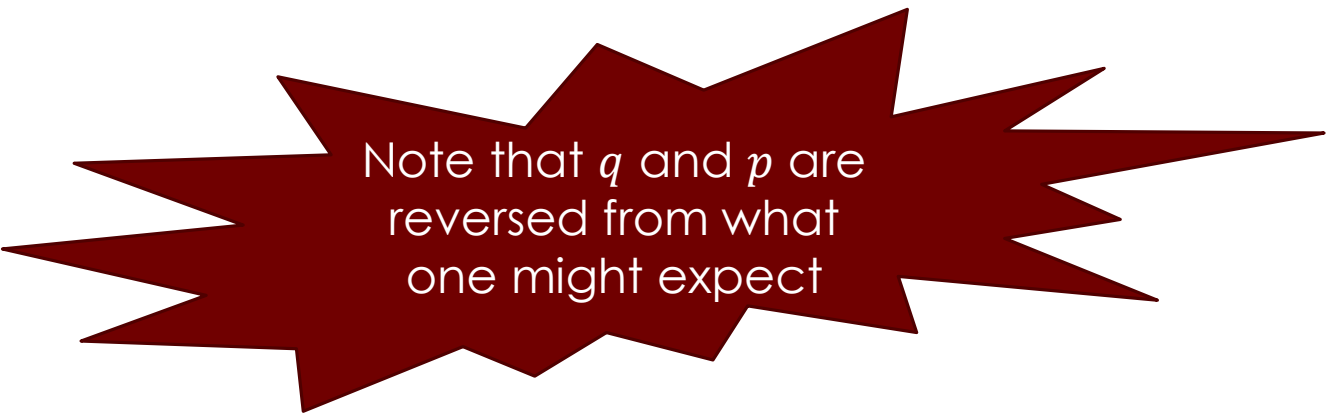
Kullback–Leibler Divergence

$D_{KL}(p||q)$ is a measure of the inefficiency of assuming that the distribution is q when the true distribution is p . For example, if we knew the true distribution p of the random variable, we could construct a code with average description length $H(p)$. If, instead, we used the code for a distribution q , we would need $H(p) + D_{KL}(p||q)$ bits on the average to describe the random variable.

Intuitively, there are three cases:

- If p is high and q is high then we are happy.
- If p is high and q is low then we pay a price.
- If p is low then we don't care (because of the expectation).

Coming Back to Variational Inference



Note that q and p are reversed from what one might expect

We want:

$$\min D_{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X}))$$

The Evidence Lower Bound (ELBO)

We actually can't minimize the KL-divergence exactly, but we can minimize a function that is equal to it up to a constant. This is the evidence lower bound (ELBO).

$$D_{KL}(q(\mathbf{Z})||p(\mathbf{Z}|\mathbf{X})) = E_q \log_2 \frac{q(\mathbf{Z})}{p(\mathbf{Z}|\mathbf{X})} = - \underbrace{\left(E_q(\log_2 p(\mathbf{Z}, \mathbf{X})) - E_q(\log_2 q(\mathbf{Z})) \right)}_{\text{ELBO}} + \log_2 p(\mathbf{X})$$

Notice that $\log_2 p(\mathbf{X})$ does not depend on q . So, as a function of the variational distribution, minimizing the KL-divergence is the same as maximizing the ELBO.

Variational Inference

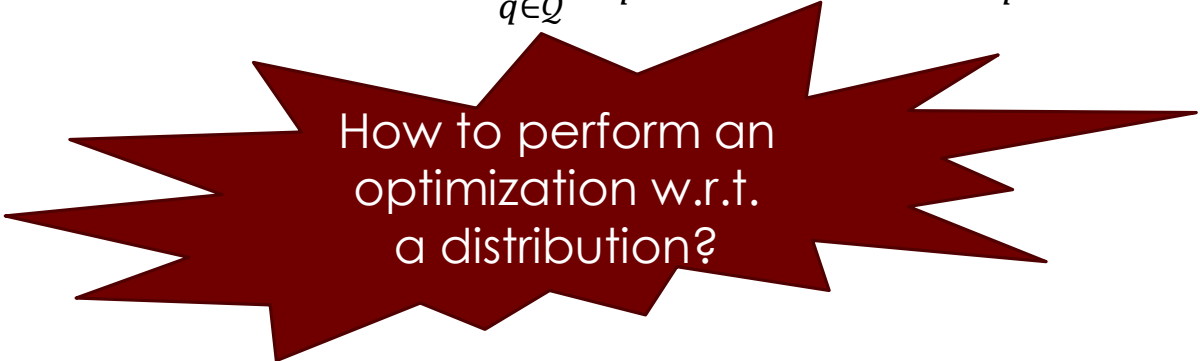
Final optimization problem:

$$\begin{aligned} & \max_{q \in \mathcal{Q}} E_q(\log_2 p(\mathbf{Z}, \mathbf{X})) - E_q(\log_2 q(\mathbf{Z})) \\ E_q(\log_2 p(\mathbf{Z}, \mathbf{X})) - E_q(\log_2 q(\mathbf{Z})) &= E_q \left(\log_2 \frac{p(\mathbf{Z}, \mathbf{X})}{q(\mathbf{Z})} \right) = E_q \left(\log_2 \frac{p(\mathbf{X}|\mathbf{Z})p(\mathbf{Z})}{q(\mathbf{Z})} \right) = \\ &= \underbrace{E_q(\log_2 p(\mathbf{X}|\mathbf{Z}))}_{\text{data term}} - \underbrace{D_{KL}(q(\mathbf{Z})||p(\mathbf{Z}))}_{\text{regularizer}} \end{aligned}$$

Variational Inference

Final optimization problem:

$$\max_{q \in \mathcal{Q}} E_q(\log_2 p(\mathbf{Z}, \mathbf{X})) - E_q(\log_2 q(\mathbf{Z}))$$



How to perform an
optimization w.r.t.
a distribution?

- Mean field approximation (factorized family)
- Parametric approximation (parametric family)

Example: Bayesian Neural Networks

Bridging Deep Learning and Probabilistic Programming

Bayesian Neural Networks



Bayesian learning for neural networks

Advisers: [Geoffrey Hinton](#)

Authors: [Radford M. Neal](#)

Publication:

· Doctoral Dissertation

Bayesian learning for neural networks

University of Toronto Toronto, Ont., Canada, Canada ©1995

ISBN:0-612-02676-0

1995 Doctoral Dissertation

 [Bibliometrics](#)

- Citation Count: 56
- Downloads (cumulative): n/a
- Downloads (12 Months): n/a
- Downloads (6 Weeks): n/a

Lecture Notes in
Statistics

118

Radford M. Neal

**Bayesian Learning
for Neural Networks**



<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.446.9306&rep=rep1&type=pdf>

Bayesian Neural Networks

A Bayesian neural network is a neural network with a prior distribution on its weights

Define the prior on the weights and biases $\mathbf{w}$ to be the standard normal:

$$p(\mathbf{w}) = \text{Normal}(\mathbf{0}, \mathbf{I})$$

Define the likelihood for a data point as:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Bernoulli}(NN(\mathbf{x}, \mathbf{w}))$$

Where $\mathbf{x} \in \mathbb{R}^d$ is the feature vector, $y \in \{0,1\}$ is the output (binary classification), and $NN(\mathbf{x}, \mathbf{w})$ is the function (deterministic) implemented by the neural network whose weights and biases form the latent variables $\mathbf{w}$. The output of the neural network must be a value between 0 and 1 (for example the last layer has only one sigmoid neuron).

Or:

$$p(y|\mathbf{x}, \mathbf{w}) = \text{Categorical}(NN(\mathbf{x}, \mathbf{w}))$$

Where $\mathbf{x} \in \mathbb{R}^d$ is the feature vector, $y \in \{1,2, \dots k\}$ is the output (multiclass classification), and $NN(\mathbf{x}, \mathbf{w})$ is the function (deterministic) implemented by the neural network whose weights and biases form the latent variables $\mathbf{w}$. The outputs of the neural network must sum to 1 (for example the last layer is a softmax layer).

Why Bayesian Neural Networks?

- **Uncertainty in predictions:** the Bayesian Neural Network informs us about the uncertainty in its predictions
- **Uncertainty in representations:** We also get uncertainty estimates of our weights which could inform us about the stability of the learned representations of the network
- **Regularization with priors:** Weights are often L2-regularized to avoid overfitting, this very naturally becomes a Gaussian prior for the weight coefficients. We could, however, imagine all kinds of other priors
- **Transfer learning with informed priors:** If we wanted to train a network on a new object recognition data set, we could bootstrap the learning by placing informed priors centered around weights retrieved from other pre-trained networks

Generating Data

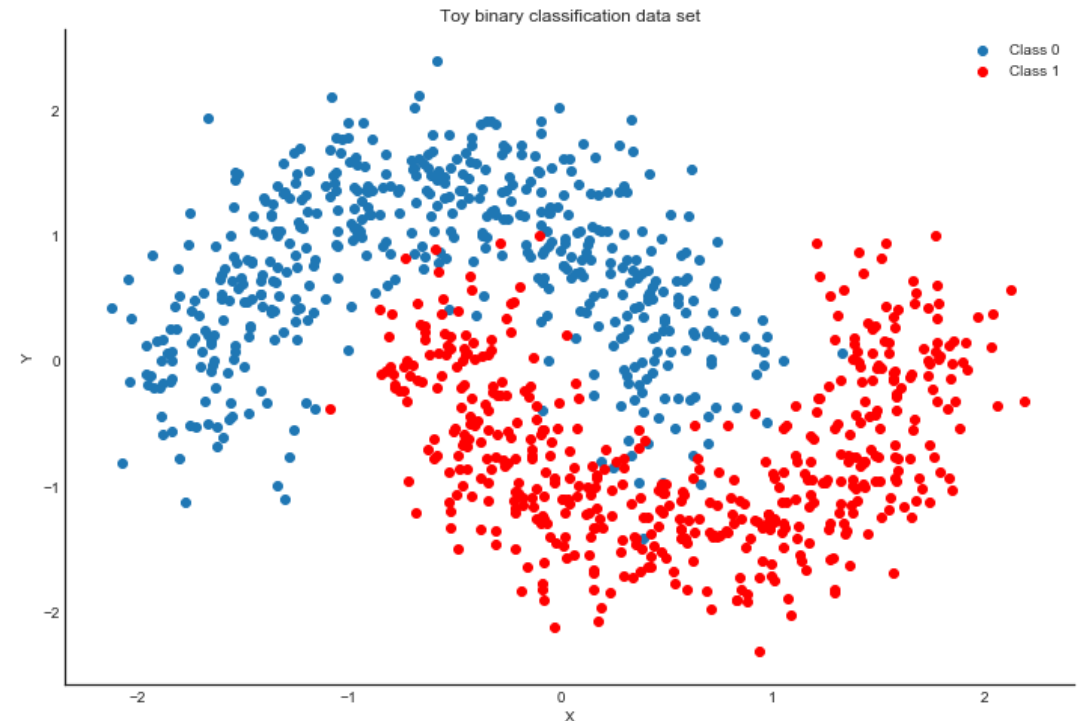
“Two Moons” dataset: toy dataset for a simple binary classification problem that's not linearly separable.

```
import numpy as np
import pymc as pm
import pytensor
import pytensor.tensor as pt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import scale

floatX = pytensor.config.floatX
RANDOM_SEED = 9927
rng = np.random.default_rng(RANDOM_SEED)

X, Y = make_moons(noise=0.2, random_state=0, n_samples=1000)
X = scale(X)
X = X.astype(floatX)
Y = Y.astype(floatX)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.5)
```



Model Specification

```
def construct_nn(ann_input, ann_output):
    n_hidden = 5

    # Initialize random weights between each layer
    init_1 = rng.standard_normal(size=(X_train.shape[1], n_hidden)).astype(floatX)
    init_2 = rng.standard_normal(size=(n_hidden, n_hidden)).astype(floatX)
    init_out = rng.standard_normal(size=n_hidden).astype(floatX)

    coords = {
        "hidden_layer_1": np.arange(n_hidden),
        "hidden_layer_2": np.arange(n_hidden),
        "train_cols": np.arange(X_train.shape[1]),
        "obs_id": np.arange(X_train.shape[0]),
    }

    with pm.Model(coords=coords) as neural_network:
        ann_input = pm.Data("ann_input", X_train, mutable=True, dims=("obs_id", "train_cols"))
        ann_output = pm.Data("ann_output", Y_train, mutable=True, dims="obs_id")

        # Weights from input to hidden layer
        weights_in_1 = pm.Normal(
            "w_in_1", 0, sigma=1, initval=init_1, dims=("train_cols", "hidden_layer_1")
        )

        # Weights from 1st to 2nd layer
        weights_1_2 = pm.Normal(
            "w_1_2", 0, sigma=1, initval=init_2, dims=("hidden_layer_1", "hidden_layer_2")
        )

        # Weights from hidden layer to output
        weights_2_out = pm.Normal("w_2_out", 0, sigma=1, initval=init_out, dims="hidden_layer_2")
```

Model Specification

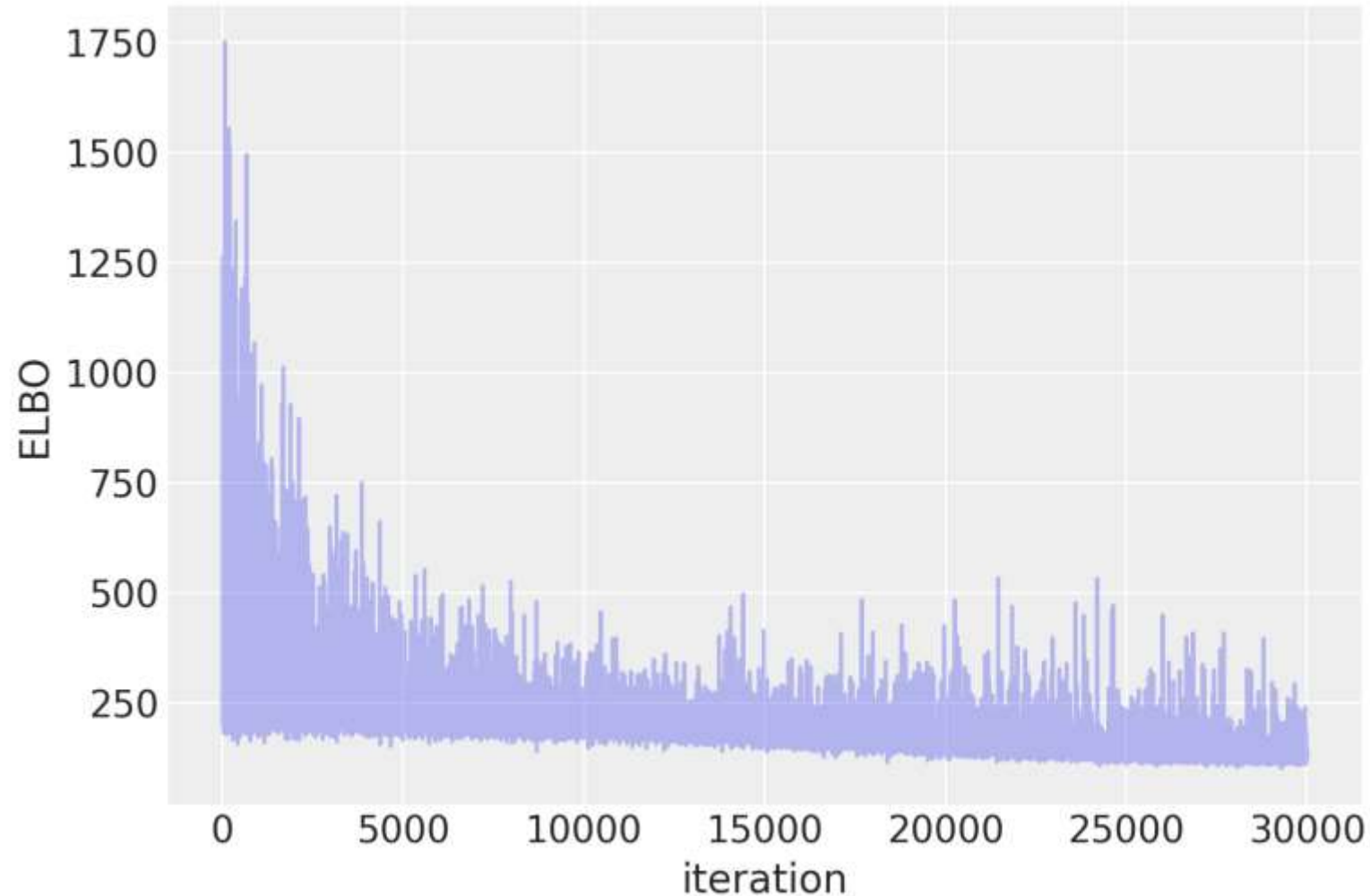
```
# Build neural-network using tanh activation function
act_1 = pm.math.tanh(pm.math.dot(ann_input, weights_in_1))
act_2 = pm.math.tanh(pm.math.dot(act_1, weights_1_2))
act_out = pm.math.sigmoid(pm.math.dot(act_2, weights_2_out))

# Binary classification -> Bernoulli likelihood
out = pm.Bernoulli(
    "out",
    act_out,
    observed=ann_output,
    total_size=Y_train.shape[0], # IMPORTANT for minibatches
    dims="obs_id",
)
return neural_network
```

```
neural_network = construct_nn(X_train, Y_train)
```

Automatic Differentiation Variational Inference (ADVI)

```
with neural_network:  
    approx = pm.fit(n=30_000)  
  
plt.plot(approx.hist, alpha=0.3)  
plt.ylabel("ELBO")  
plt.xlabel("iteration")
```



Prediction

- We can use `sample_posterior_predictive()` to generate new data (in this case class predictions) from the posterior (sampled from the variational estimation)

```
trace = approx.sample(draws=5000)

with neural_network:
    pm.set_data(new_data={"ann_input": X_test})
    ppc = pm.sample_posterior_predictive(trace)
    trace.extend(ppc)
```

- We can average the predictions for each observation to estimate the underlying probability of class 1

```
pred = ppc.posterior_predictive["out"].mean(("chain", "draw")) > 0.5
print(f"Accuracy = {(Y_test == pred.values).mean() * 100}%")    # output: Accuracy = 95.0%
```